An Improved RRT* Path Planning Algorithm in Dynamic Environment

Jianyu Li, Kezhi Wang, Zonghai Chen, and Jikai Wang(✉)

University of Science and Technology of China, Hefei, Anhui, China
wangjk@ustc.edu.cn

Abstract. With the increasingly diverse usage scenarios of mobile robots, the path planning of mobile robots in dynamic environments has become a hot issue. Aiming at the slow planning speed of RRT* algorithm in dynamic environment, this paper proposes an improved RRT* algorithm: efficient dynamic rapidly-exploring random tree star (ED-RRT*). In a static environment, the algorithm uses goal-biased sampling to reduce the randomness of the RRT* sampling method, and sets the sampling step size to be adaptive to improve the planning speed of the initial path. When the dynamic environment encounters obstacles and needs to be re-planned, the local target points are given based on the initial path and environmental information to quickly complete the local re-planning. The path information planned in the static environment is used as the prior information to improve the efficiency of the replanning algorithm. Finally, in the simulation environment based on OpenCV2, the ED-RRT* algorithm is compared with the improved RRT* algorithm. The experimental results show that ED-RRT* has faster dynamic performance and fewer nodes.

Keywords: Mobile robot · Dynamic environment · Path planning · RRT*

1 Introduction

In recent years, Mobile robots have been applied in many fields, such as Search-and-Rescue [1], healthcare services [2], autonomous delivery [3], and so on. Path planning has always been one of the hot issues in the field of robotics. In static environment, there are already many typical algorithms like A* [4], Dijkstra's algorithm [5], improved A* [6]. However, due to the fact that obstacle movement is irregular in dynamic environment, it is still challenging to plan a collision-free, safe, efficient and high-quality path.

Because of the speed and robustness in finding the path to the target, sampling-based planning algorithms are popular global path planning approaches. RRT [7], which quickly explores the configuration space through random sampling, has high computational efficiency and probabilistic completeness. Although the generality of the RRT algorithm can solve the path planning problem of the robot in different dimensions, it cannot guarantee asymptotic optimality. In environments with narrow channels, the convergence rate of the RRT algorithm is slow.

W. Fan et al. (Eds.): AsiaSim 2022, CCIS 1713, pp. 301–313, 2022.
https://doi.org/10.1007/978-981-19-9195-0_25

In order to solve the shortcomings of the traditional RRT algorithm, scholars have made improvements to the RRT algorithm. The RRT-connect algorithm [8] is an algorithm based on the RRT algorithm, which simultaneously grows two rapid-exploration random trees from the starting point and the ending point to search the state space. Bidirectional rapid-exploration random trees speed up algorithm convergence. Urmson and Simmons [9] proposed a heuristic function that makes the expansion of random trees biased to speed up the convergence of the algorithm. Karaman and Frazzoli proposed the RRT* algorithm [10] to find the optimal path of the RRT algorithm. By reselection of parent nodes and rewire function, RRT* algorithm can find an optimal or near-optimal path. Rapidly-exploring random trees star fixed nodes (RRT*FN) [11] proposed by reduces the memory usage of the computing center by limiting the maximum number of nodes in the tree. In order to solve the problem of large RRT* sampling space and long sampling time, Gammell and Srinivasa proposed the Informed-RRT* algorithm [12]. The algorithm generates an ellipse sampling space determined by the starting point, the target point, and the current path length, and by heuristic sampling in this ellipse region, it accelerates the convergence to the optimal solution.

Although the algorithms mentioned above have been optimized to a certain extent, none of them utilizes environmental information. In order to introduce environmental information to guide RRT, some researchers have explored the method of combining RRT with artificial potential field [13]. The algorithm switches to RRT planning when the artificial potential field method falls into a local minimum, and switches back to the artificial potential field method when it jumps out of the local minimum. The smart rapidly-exploring random tree star (RRT*-Smart) [14] has made improvements on the basis of RRT*, mainly by optimizing the path. RRT*-Smart is exactly the same as RRT* in the first stage, but after finding a feasible path from the start point to the end point it starts to optimize the path. The optimization process starts from the leaf node, constantly looking for whether it can directly connect to the parent node without collision. If it can be directly connected, there will be one more straight line and one less curve. These algorithms implicitly utilize environmental information, but they are improved based on RRT or RRT*. They are dedicated to searching for the optimal path and have poor real-time performance, so they are not suitable for dynamic path planning with high real-time requirements.

Previous work on re-planning is incremental replanning algorithm, such as D* [15] and D* Lite [16]. The D* Lite algorithm is a reverse search. At first, according to the known environmental information, the unknown part is regarded as a free space, and the global optimal path from the target point to the starting point is planned. At this time, a path field is established to provide the basis for the optimal approach to the target point incrementally. The D* Lite algorithm can be well applied to unknown environment for path planning. Due to the idea of incremental planning, it has fewer re-planning times. However, when the space is relatively large, the number of grid nodes to be maintained in the reverse search process increases sharply, which increases the time complexity of the search. Qi et al. [17] proposed a real-time dynamic path planning method combining artificial potential field method and goal-biased RRT algorithm. The algorithm offsets the random sampling points towards the target point, uses the constructed gravitational

potential field and repulsive potential field to make the random tree approach the target while staying away from obstacles, and has an adaptive growth step in different local environments. It greatly reduces the repeated calculation of the algorithm in the local minimum region, and uses a local re-planning strategy for dynamic environments. However, this algorithm still has the problem of long search time and long replanning path.

In terms of above issues, we propose an RRT*-based algorithm termed as ED-RRT* algorithm. ED-RRT* consists of two stages, the first stage is the global path planning, and the second stage is the re-planning for the dynamic environment. In the first stage, the idea of goal-biased sampling is used to make the sampling points converge quickly. This ensures that a safe path can be planned as an initial path in a relatively short time. When the robot moves according to the global path and encounters dynamic obstacles, the algorithm will enter the re-planning stage. Taking points on the global path as new target points, replanning is performed within a certain distance to avoid collisions.

To summarize, the main contributions of this work are as follows:

(1) **Global Path Planning:** The initial environment is regarded as a static environment for initial planning, and goal-biased sampling is introduced to reduce the randomness of the RRT* algorithm. Apply an adaptive step size method to increase exploration efficiency. Such a sampling strategy can not only guide the random tree to grow toward the target, but also has the characteristic of preferentially growing a branch, which is beneficial to improve efficiency and quickly plan a path.
(2) **Path Re-planning:** The local replanning strategy is used to further improve the real-time performance of the algorithm in dynamic environments. The robot senses the movement of obstacles through sensors. When re-planning is required, the initially planned path is used as a path cache, and a certain node of the initial path is selected as a new target point to re-plan locally.

The remainder of the paper is organized as follows. The related work is reviewed in Sect. 2. An overview of ED-RRT* algorithm is provided in Sect. 3. Section 4 compares the ED-RRT* algorithm with other method under simulated scenarios. Section 5 presents our conclusions.

2 Related Work

2.1 Task Definition

Denote S as the configuration space, which represents the space where the mobile robot needs to do path planning. The space that collides with obstacles is defined $S_{obs} \subset S$, and the space that does not collide with obstacles is defined as free space $S_{free} = S \backslash S_{obs}$, Let the starting position of the mobile robot be $S_{start} \in S_{free}$ and the target area be $S_{goal} \subset S_{free}$. Therefore, the feasible path between the starting position and the target area can be defined as a set $\sigma : [0, 1] \rightarrow S_{free}$ with $\sigma(0) = S_{start}$ and $\sigma(1) \in S_{goal}$.

The path cost is defined as $c : S_{free} \rightarrow R$, where R represents the set of non-negative real numbers. Finding a path with a lower path cost is a goal of path planning, and the

minimum path cost function σ^* is defined as follows:

$$\sigma^* = \arg\min_{\sigma \in S} \{c(\sigma) | \sigma(0) = S_{start}, \sigma(1) = S_{goal}, \forall x \in [0, 1], \sigma(x) \in S_{free}\} \quad (1)$$

2.2 RRT* Algorithm

Below is the pseudo code for RRT*.

Algorithm 1: RRT* Algorithm

```
Input: S, S_start, S_goal
Output: A path σ from S_start to S_goal
initializeTree(S)
insertRootNode()
for i = 1 to n do
    q_rand ←SampleRandomNode(S)
    q_nearest ←FindNearestNeighbor()
    q_new ←Steer(q_nearest, q_rand, stepsize)
    if ObtacleFree(q_nearest, q_new)
        q_min ←SelectParent()
        InsertNode(q_min, q_new)
        Rewire(Q_near, q_min, q_new)
```

RRT* algorithm is much similar in comparison to RRT. RRT* starts by building a tree using random samples from the robot's operating space and adds new samples to the tree. However, there are two notable differences with respect to RRT: new edge addition and additional step to optimize path cost through rewiring.

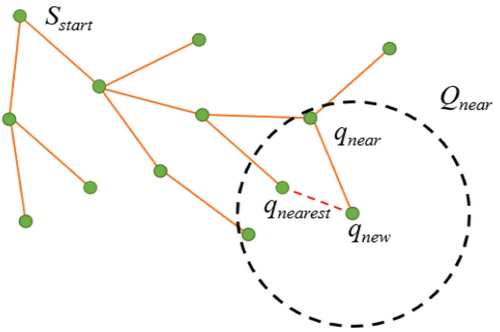


Fig. 1. Node growth progress of RRT*

Figure 1 shows the process of adding a new node in the RRT* algorithm. After RRT* finds the node $q_{nearest}$ closest to q_{new}, it does not immediately add edge ($q_{nearest}$, q_{new}) to the expansion tree, but takes q_{new} as the center and r as the radius to find all potential parent node sets. As shown by the dotted circle in the figure, Q_{near} represents the search area to find potential parent nodes. For every potential parent node in the boundary, as shown by the red dotted line in the figure, a path is drawn starting from the newly added node to the nearby node. If the path is obstacle free and the total cost of this path is lower than the cost of current path, the previous edge in the tree will be deleted and the new edge will be added in the tree.

3 ED-RRT* Path Planning Algorithm

Global path planning is an improved RRT* algorithm that combines goal-biased sampling and adaptive step size. Global path planning is an improved RRT* algorithm that combines biased sampling and adaptive step size. Compared with the RRT* algorithm, bias sampling is introduced to guide the growth direction of the tree, thereby saving the time spent on path planning. Change the fixed step size to an adaptive step size, which can increase the efficiency of exploring the environment. In order to adapt to the dynamic environment, it is important to improve the planning efficiency.

3.1 ED-RRT* Algorithm Framework

ED-RRT* runs three threads for global path planning, local path replanning and visualization. The description and relationship of these three threads are shown in the following figure (Fig. 2):

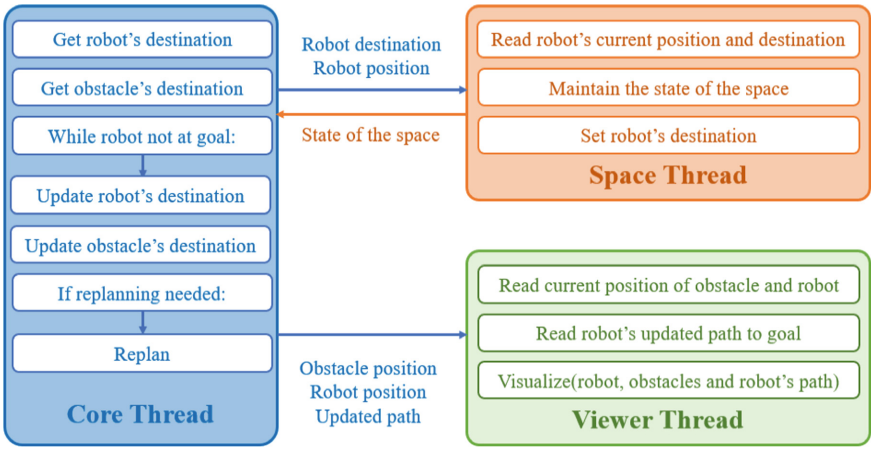


Fig. 2. Framework of the ED-RRT* algorithm

After the initial path is planned, space thread is responsible for maintaining the state information in the environment and reading the current position and target of the robot.

At the same time, it also needs to let the robot drive according to the planned path. Core thread needs to obtain the position and movement of the robot and obstacles in real time.

If the robot does not reach the goal, it is necessary to determine whether the robot will collide with a dynamic obstacle before the next node along the current path. When there is a possibility of collision, it will enter the re-planning stage. Viewer thread is a simulation and visualization tool for the ED-RRT* algorithm, which obtains the position of the robot, the position of obstacles and the path of the robot from the core thread. After rendering, they are presented in the simulation environment built by this article.

3.2 Goal-Biased Sampling

The generation of nodes in the RRT* algorithm is completely random, which causes the exploration process to be non-directional. In turn, a lot of time is spent on path exploration in invalid areas. The sampling method of goal-bias can guide the growth direction of the tree, so that the exploration process is carried out towards the goal point. The generation probability formula for each direction of random nodes is as follows:

$$q_{rand} = \begin{cases} \mu \times q_{goal} + (1-\mu) \times q_{rand} & obstacle = 0,\ n > \alpha \\ q_{goal} & obstacle = 0,\ n \leq \alpha \\ q_{rand} & obstacle = 1 \end{cases} \tag{2}$$

When there is no obstacle nearby, that is, when obstacle = 0, the random nodes will tend to be chosen as points near the goal point. This tendency depends on the size of μ, and the larger the μ is, the higher the tendency will be. n represents a random number, and α represents the probability of directly selecting the goal point as a random node. When obstacles are detected, new nodes are randomly generated.

3.3 Adaptive Step Size

The choice of step size is important to the efficiency of the algorithm. If the step size is too small, the number of steps to be extended is large, and the overall efficiency of the algorithm decreases. If the step size is too large, the sampling success rate is low, which will reduce the performance of obstacle avoidance and the efficiency of the algorithm. The following formula describes the choice of step size:

$$\tau = \begin{cases} \lambda & d \leq 0.75 d_s \\ 1.5\lambda & 0.75 d_s < d \leq d_s \\ 2\lambda & d > d_s \end{cases} \tag{3}$$

In the above formula, d is the distance from nearby obstacles, d_s is the set safety distance, and λ is the preset step size. When the expansion tree is far away from the obstacle, a fixed step is used for expansion to speed up the convergence speed of the algorithm; when the expansion tree is expanded to the vicinity of the obstacle, a small step is used for expansion.

3.4 Path Re-planning

Re-planning begins with the determination of whether or not the moving obstacle is blocking the path. In the detection range of the sensor, the method of judging whether the dynamic obstacle will block the path is shown in Fig. 3.

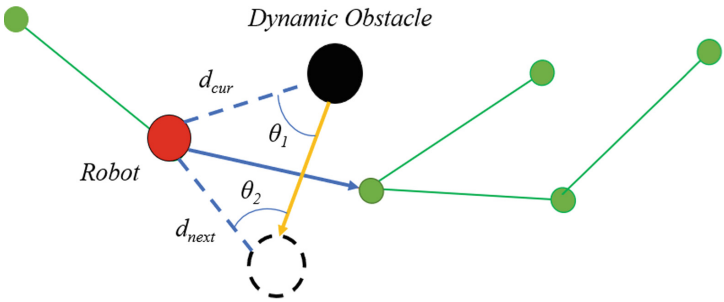


Fig. 3. Schematic diagram of judging whether dynamic obstacles block the path (Colour figure online)

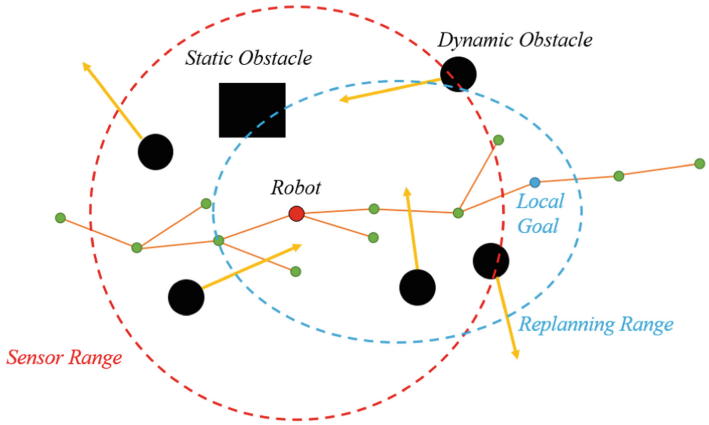
In Fig. 3, the red dots represent the current position of the robot. The black point represents the current position of the dynamic obstacle. At the next moment, the robot will follow the blue trajectory to the next green node, and the dynamic obstacle will follow the yellow trajectory to the position of the dotted circle. The distance between the robot's current moment and the dynamic obstacle is d_{cur}. The distance between the position of the robot at the current moment and the dynamic obstacle at the next moment is d_{next}. θ_1 and θ_2 respectively represent the angle between the two lines.

First, it is judged whether the distance between d_{cur} and d_{next} is less than the safe distance d_{safe}. If so, it is considered that a collision will occur. If not, judge the magnitude of angles θ_1 and θ_2. When the angles of A and B are both less than 90°, dynamic obstacles cross the robot's path. In this case we think there will also be a collision. When there is a possibility of a collision, the algorithm re-plans the path.

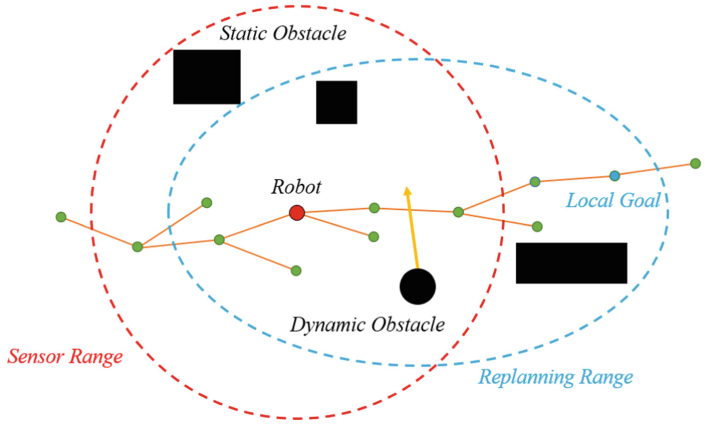
The replanning process is divided into the following three steps:

(1) **Set Up Neighborhood for Replanning:** Any sensor has a corresponding sensing range. In the ED-RRT* algorithm, a circle with robot as the center and radius R simulates the range covered by the sensor. If there are many dynamic obstacles in the sensing range, the neighborhood for replanning will be smaller, otherwise it will be larger.

(2) **Set Up a Local Goal:** Using the initial planned path can save the time of re-planning, so when re-planning, only the nodes on the initial path are selected as the local goal. Figure 4 shows the setting of replanning regions and local goals under different numbers of dynamic obstacles. When there are many dynamic obstacles around the robot, the local goal will be set closer to the robot. If the re-planned



(a) In scenarios with many dynamic obstacles



(b) In scenarios with few dynamic obstacles

Fig. 4. In different scenarios, the selection of neighborhood and local goal

path is too long, it will still be blocked by other dynamic obstacles and needs to be re-planned.

(3) **Rewire:** After the local goal is established, the tree will be pruned locally. ED-RRT* algorithm will plan a new local path according to the location of the local goal. After the robot reaches the local goal, it still follows the initial path. As shown in Fig. 5, the green dotted line is part of the initial path. When replanning, this part of the path is discarded, and the blue path is replanned to reach the local goal. Compared with direct re-planning, this local re-planning utilizes the initial path information to save computation time.

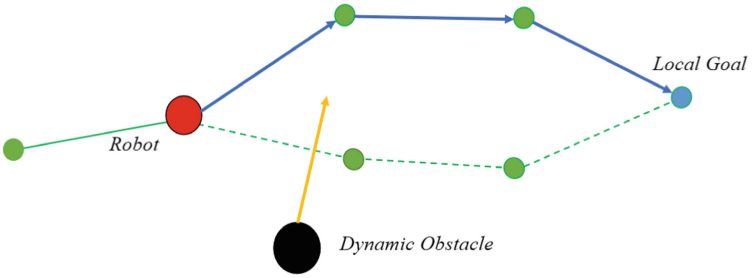
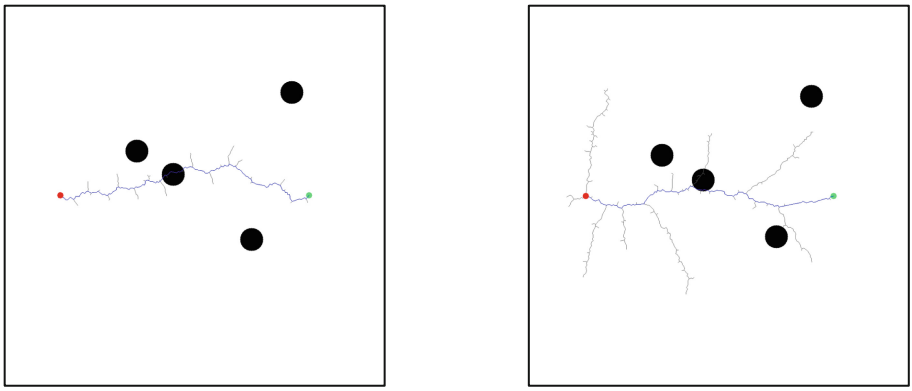


Fig. 5. The rewire progress of ED-RRT* (Colour figure online)

4 Experimental Results

This paper builds a simulation environment based on OpenCV2 to realize the simulation and visualization of the path planning algorithm. We compare our algorithm with improved RRT* algorithm in three different scenarios, and the results are shown in

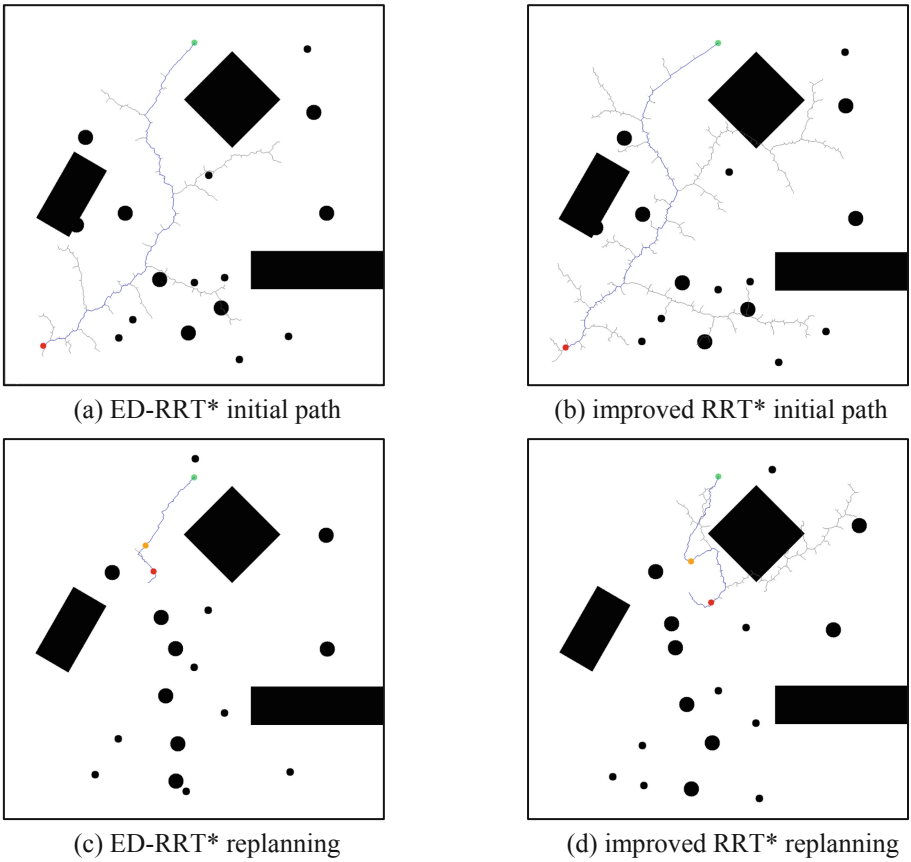


(a) ED-RRT* initial path (b) improved RRT* initial path

(c) ED-RRT* replanning (d) improved RRT* replanning

Fig. 6. The performance of the two algorithms in the scenario with a few dynamic obstacles (Colour figure online)

Fig. 6, Fig. 7 and Fig. 8. The red dots in these figures represent the mobile robot, the black circles represent dynamic obstacles, the yellow dots represent the local goal, the gray track represents the RRT, and the blue track represents the planned path.



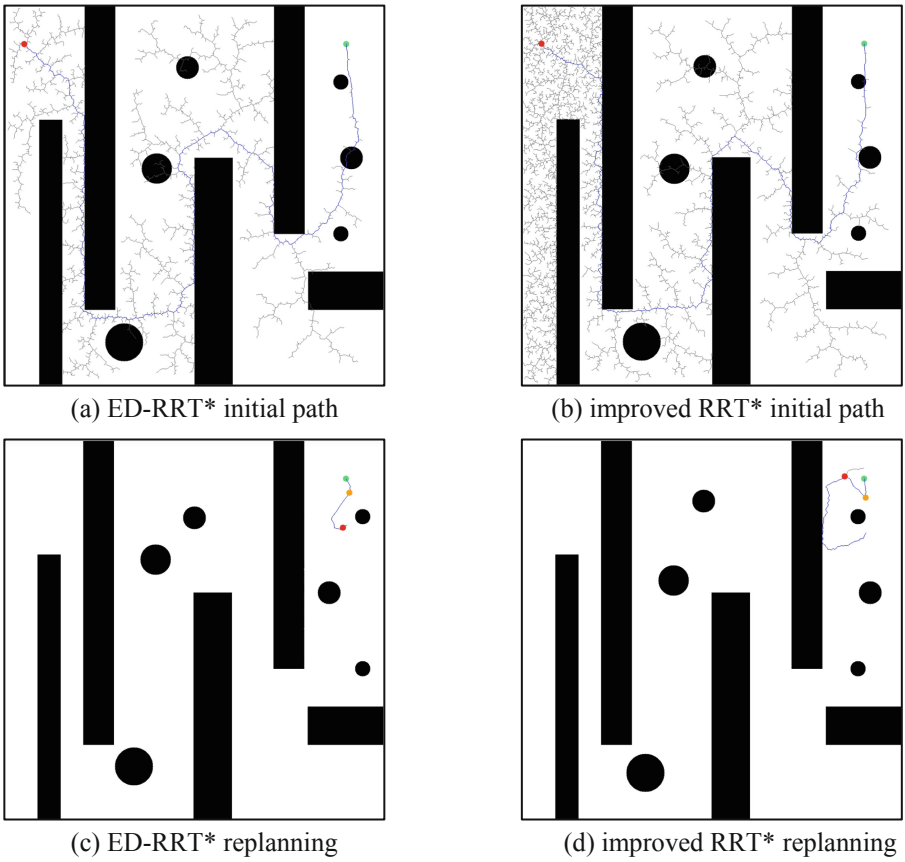
(a) ED-RRT* initial path (b) improved RRT* initial path

(c) ED-RRT* replanning (d) improved RRT* replanning

Fig. 7. The performance of the two algorithms in the scenario with many dynamic obstacles (Colour figure online)

The obstacles are expanded in the simulation environment, and the robot can be regarded as a particle to simplify the path planning process. Since dynamic obstacles are not treated as obstacles in the initial path planning, the initial path may pass through dynamic obstacles. As shown in Fig. 6, when the number of dynamic obstacles in the environment is small, the initial path lengths planned by ED-RRT* and improved RRT* are similar, but the number of nodes generated by ED-RRT* exploration is significantly smaller. During the replanning phase, both algorithms perform well.

As shown in Fig. 7, the initial path planning in the scenario with many dynamic obstacles also obtains the same results as the simple scenario. But when replanning, ED-RRT* plans shorter paths and explores fewer nodes.



(a) ED-RRT* initial path (b) improved RRT* initial path

(c) ED-RRT* replanning (d) improved RRT* replanning

Fig. 8. The performance of the two algorithms in a maze scenario with dynamic obstacles (Colour figure online)

As shown in Fig. 8, in a maze scenario with dynamic obstacles, improved RRT* needs to expand more nodes to explore feasible paths. Since ED-RRT* utilizes the goal-biased sampling method, it will explore towards the target point. In the re-planning stage, ED-RRT* can also plan a shorter path and improve the operating efficiency of the algorithm.

Table 1 presents the average path lengths and average running times of ED-RRT* and improved RRT* under the three scenarios, respectively.

As shown in Table 1, in Scenario 1, the average path length planned by ED-RRT* is 7.9% shorter and the running time is 9.4% shorter than improved RRT*. In Scenario 2, the average path length planned by ED-RRT* is 21.2% shorter and the running time is 24.7% shorter than improved RRT*. in Scenario 3, the average path length planned by ED-RRT* is 16.0% shorter and the running time is 20.7% shorter than improved RRT*.

Table 1. Comparison of the average path length and average running time of the two algorithms

Scenario	Algorithm	Path Length	Running time/s
Scenario 1	ED-RRT*	58	8.51
	Improved RRT*	63	9.39
Scenario 2	ED-RRT*	89	14.78
	Improved RRT*	113	19.62
Scenario 3	ED-RRT*	237	28.05
	Improved RRT*	282	35.37

5 Conclusion

Path planning in dynamic scenarios is one of the key issues in robotics research. Due to its randomness, the sampling-based algorithm will lead to an increase in the amount of computation, and it performs poorly in dynamic scenarios. Based on the RRT* algorithm, the ED-RRT* proposed in this paper introduces goal-biased sampling, adaptive step size and local reprogramming. The goal-biased sampling guides the exploration direction. The adaptive step size method accelerates the efficiency of exploration, and re-planning in local sub-regions increases the dynamic performance of the algorithm. Compared with improved RRT*, this algorithm has higher efficiency in dynamic scenarios.

Although the algorithm has relatively good performance, it is necessary to increase the trajectory optimization part of the back-end for practical problem, so that the trajectory can meet the kinematic constraints of the mobile robot. This is also something to consider in future work.

Acknowledgments. This work was supported by the Natural Science Found of China (Grant No. 62103393).

References

1. Punith, K.M.B., Sumanth, S., Savadatti, M.A.: Internet rescue robots for disaster management. Int. J. Wirel. Microwave Technol. **11**(2), 13–23 (2021)
2. Vasquez, A., et al.: Deep detection of people and their mobility aids for a hospital robot. In: 2017 European Conference on Mobile Robots (ECMR). IEEE (2017)
3. Alatise, M.B., Hancke, G.P.: A review on challenges of autonomous mobile robot and sensor fusion methods. IEEE Access **8**, 39830–39846 (2020)
4. Hart, P.E., et al.: A formal basis for the heuristic determination of minimum cost paths. IEEE Trans. Syst. Sci. Cybern. **4**(2), 100–107 (2007)
5. Dijkstra, E.W.: A note on two problems in connexion with graphs. Numer. Math. **1**(1), 269–271 (1959)
6. Li, Z., Shi, R., Zhang, Z.: A new path planning method based on sparse A* algorithm with map segmentation. Trans. Inst. Meas. Control. **44**(4), 916–925 (2022)
7. LaValle, S.M., Kuffner, J.J.: Randomized kinodynamic planning. Int. J. Robot. Res. **20**(5), 378–400 (2001)

8. Kuffner, J.J., LaValle, S.M.: RRT-connect: an efficient approach to single-query path planning. In: Proceedings of the 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No. 00CH37065), vol. 2, pp. 995–1001 (2000)
9. Urmson, C., Simmons, R.: Approaches for heuristically biasing RRT growth. In: Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems. IEEE, Las Vegas, USA, vol. 2, pp. 1178–1183 (2003)
10. Karaman, S., Frazzoli, E.: Sampling-based algorithms for optimal motion planning. Int. J. Robot. Res. **30**(7), 846–894 (2011)
11. Adiyatov, O., Varol, H.A.: Rapidly-exploring random tree based memory efficient motion planning. In: 2013 IEEE International Conference on Mechatronics and Automation, pp. 354–359. IEEE (2013)
12. Gammell, J.D., Srinivasa, S.S., Barfoot, T.D.: Informed RRT*: optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic. In: Proceedings of the 2014 IEEE/RSJ International Conference on Intelligent Robots and Systems. IEEE (2014)
13. Fang, Z., Qidan, Z., Guoliang, Z.: Path optimization of manipulator based on the improved rapidly-exploring random tree algorithm. J. Mech. Eng. **47**(11), 30–35 (2011)
14. Nasir, J., Islam, F., Malik, U., et al.: RRT*-smart: a rapid convergence implementation of RRT*. Int. J. Adv. Rob. Syst. **10**(7), 299–311 (2013)
15. Stentz, A., et al.: The focussed d∗ algorithm for real-time replanning. IJCAI **95**, 1652–1659 (1995)
16. Koenig, S., Likhachev, M.: Improved fast replanning for robot navigation in unknown terrain. In: Proceedings 2002 IEEE International Conference on Robotics and Automation (Cat. No.02CH37292), vol. 1, pp. 968–975 (2002)
17. Xi, Y., et al.: A real-time dynamic path planning method combining artificial potential field method and biased target RRT algorithm. J. Phys: Conf. Ser. **1**, 2021 (1905)